

500 HTTP Status Code in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

500 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss **how to Return the 500 Internal Server Error HTTP Status Code in ASP.NET Core Web API** Applications with Examples. Please read our previous article on [Configuring the Allowed HTTP Methods Globally in ASP.NET Core Web API](#) with Examples.

What is a 500 HTTP Status Code?

The HTTP Status Code 500, also known as “Internal Server Error,” is a generic error message indicating that the server encountered an unexpected condition that prevented it from fulfilling the request. This error is a server-side issue, meaning the problem is not with the client’s request but with the server’s ability to complete it.

What are the Common Causes of a 500 HTTP Status Code in ASP.NET Core Web API?

A 500 Internal Server Error means that an unexpected server-side issue occurred, preventing the API from processing the request. The following are some of the Common Causes of 500 HTTP Status Code in ASP.NET Core Web API

- **Unhandled Exceptions:** Any exception in your application that is not caught and handled can result in a 500 error. These could be due to bugs, such as null reference exceptions, divide-by-zero errors, data type mismatches, or issues with third-party libraries.
- **Configuration Issues:** Problems with the application’s configuration files or environment settings, such as incorrect settings in the appsettings.json file or other configuration files, can lead to situations where the server can’t execute as expected. For example, incorrect logging paths, missing service registrations in dependency injection, or misconfigured environments.
- **Database Errors:** Problems connecting to the database or issues with executing queries can lead to a 500 error. This could be due to incorrect connection strings, the database server being down, invalid SQL queries, or query timeouts.
- **Middleware Issues:** If there is a problem in the middleware pipeline that is not properly handled, it can cause a 500 error. This could be errors in custom middleware or issues in the configuration of built-in middleware.
- **Third-Party Services:** Failures in external APIs or services that the application relies on can result in 500 Internal Server Error.
- **Resource Exhaustion:** The server running out of resources such as memory or disk space can also lead to 500 errors.

How to Return a 500 HTTP Status Code in ASP.NET Core Web API?

In ASP.NET Core Web API, we can return a 500 Internal Server Error Status Code from our controller actions to indicate that something went wrong on the server side. This is useful when we need to handle exceptions or situations where the server cannot process the request due to an error. Returning a 500 HTTP Status Code in an ASP.NET Core Web API can be done in several ways. Let us proceed and understand the different mechanisms to return the 500 HTTP Status Code.

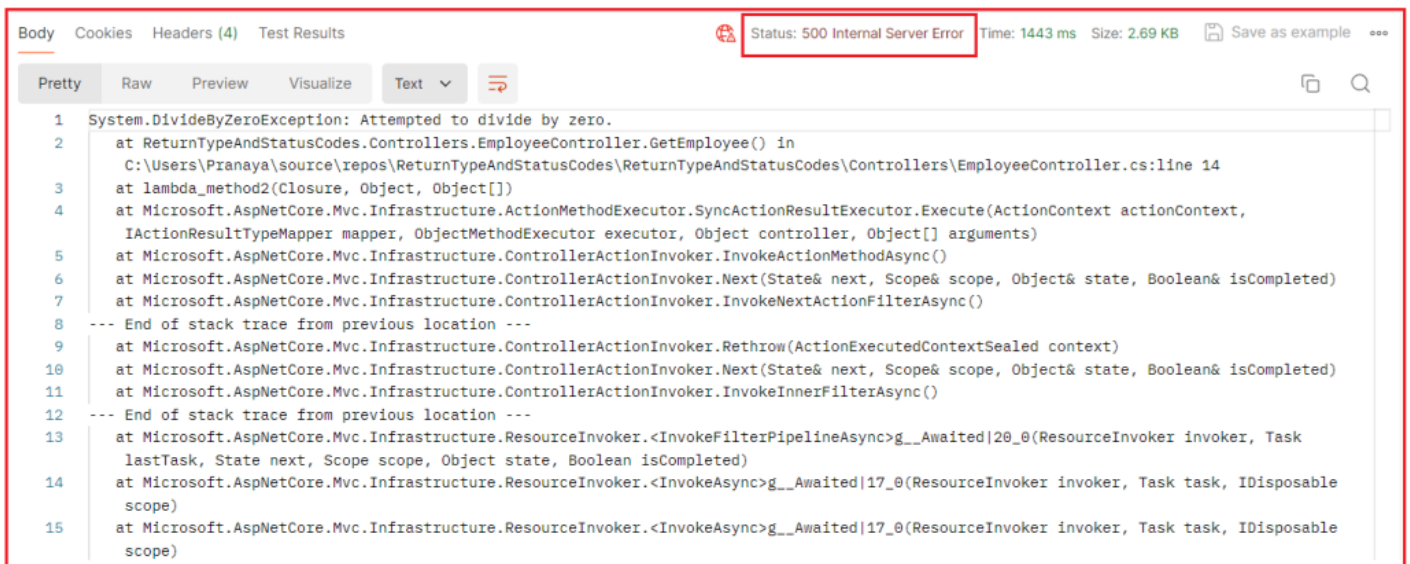
Throwing an Exception

ASP.NET Core will automatically return a 500 status code when an unhandled exception occurs. For a better understanding, please modify the Employee Controller as follows. Here, from the action method, we are throwing an exception:

```
using Microsoft.AspNetCore.Mvc;

namespace ReturnTypeAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        public IActionResult GetEmployee()
        {
            // Your application logic goes here
            int x = 10, y = 0;
            int z = x / y; //This statement will throw exception
            return Ok();
        }
    }
}
```

Now, run the application and access the above endpoint. It should give you the following 500 HTTP Error Response. This error response does not look good, and for security reasons, we shouldn't expose the actual error details to the user.



Using StatusCode Method to Return 500 Internal Server Error in ASP.NET Core Web API:

The StatusCode method allows you to return a specific HTTP status code. For example, you can use it in your controller action to return a 500 status code with an additional optional error response in the body. Please modify the Employee Controller as follows. Here, we have written the code in such a way that at runtime, it will throw a Divide By Zero Exception.

```
using Microsoft.AspNetCore.Mvc;

namespace ReturnTypeAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        public IActionResult GetEmployee()
        {
            try
            {
                // Your logic here
                int x = 10, y = 0;
                int z = x / y; //This statement will throw exception
                return Ok();
            }
            catch (Exception ex)
            {
                // Log the exception details

                var customResponse = new
                {
                    Code = 500,
```

```

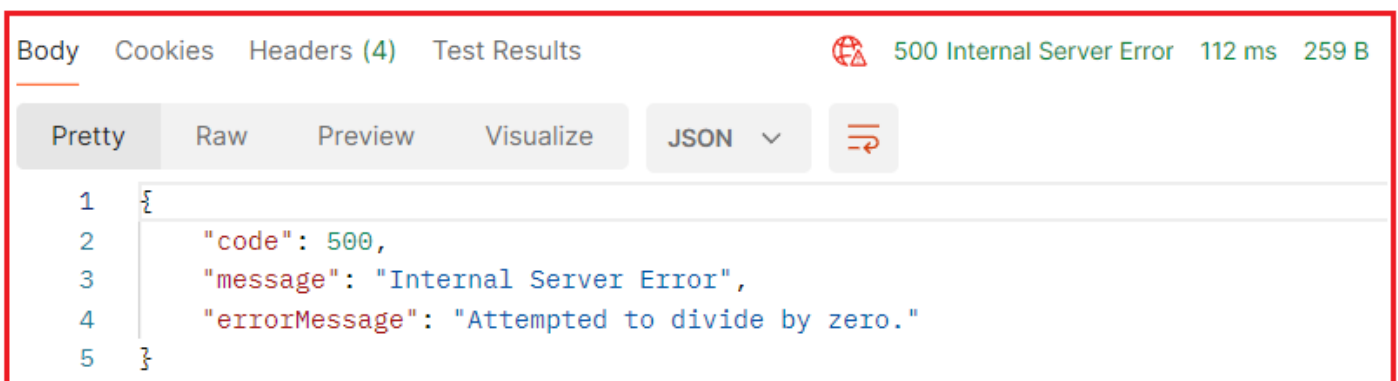
        Message = "Internal Server Error",

        // Do not expose the actual error to the client
        ErrorMessage = ex.Message
    };

    return StatusCode(StatusCodes.Status500InternalServerError,
customResponse);
    }
}
}
}
}

```

Now, if you access the above endpoint, you will get the following response:



Using ObjectResult to Return 500 Internal Server Error:

You can use the `ObjectResult` class to return a 500 status code with a custom message. This is useful if you want to return additional information about the error. For a better understanding, please modify the Employee Controller as follows:

```

using Microsoft.AspNetCore.Mvc;

namespace ReturnTypeInfoAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        public IActionResult GetEmployee()
        {
            try
            {
                // Your logic here
                int x = 10, y = 0;
            }
            catch (Exception ex)
            {
                // Handle the exception and return a 500 status code
                return StatusCode(500, new { message = "Internal Server Error", errorMessage = ex.Message });
            }
        }
    }
}

```

```

        int z = x / y; //This statement will throw exception
        return Ok();
    }
    catch (Exception ex)
    {
        // Log the exception details

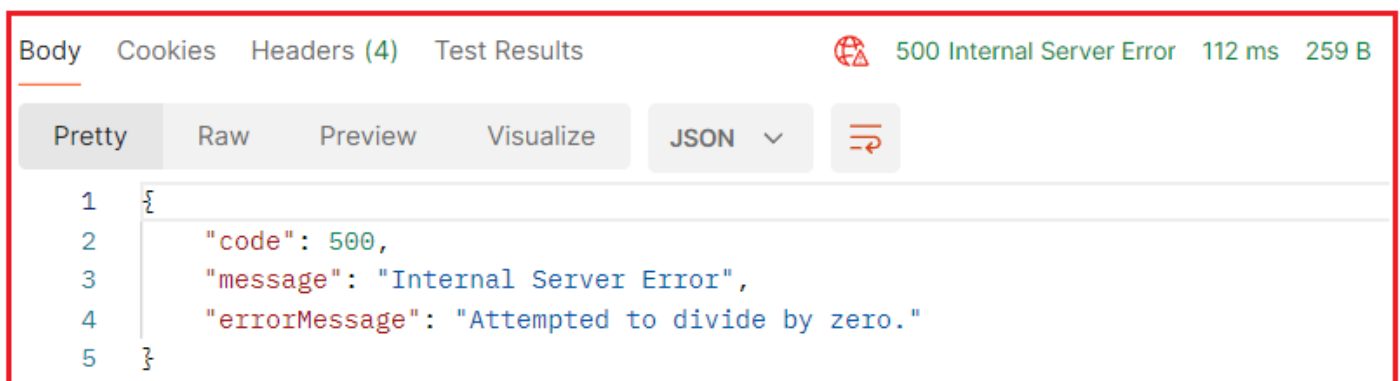
        var customResponse = new
        {
            Code = 500,
            Message = "Internal Server Error",

            // Do not expose the actual error to the client
            ErrorMessage = ex.Message
        };

        return new ObjectResult(customResponse)
        {
            StatusCode = StatusCodes.Status500InternalServerError
        };
    }
}
}
}
}

```

Now, run the application, access the above endpoint, and you will get the same 500 response as the previous example, as shown in the below image:



Custom Middleware for Global Exception Handling in ASP.NET Core Web API:

A more robust approach is to set up global exception-handling middleware to handle all unhandled exceptions and return a 500 status code. So, create a class file named **ErrorHandlingMiddleware.cs** within the Models folder and copy and paste the following code. The following middleware catches any unhandled exceptions thrown during the

processing of a request and returns a 500 Internal Server Error response. The following code is self-explained, so please read the comment lines for a better understanding:

```
// Importing System.Text.Json namespace for JSON serialization
using System.Text.Json;

namespace ReturnTypesAndStatusCodes.Models
{
    // Define a middleware class for error handling
    public class ErrorHandlingMiddleware
    {
        // Delegate representing the next middleware in the pipeline
        private readonly RequestDelegate _next;

        // Constructor to initialize the middleware with the next delegate in the
        pipeline
        public ErrorHandlingMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        // Method to invoke the middleware logic
        public async Task Invoke(HttpContext context)
        {
            try
            {
                // Try to process the request by passing it to the next
                middleware

                await _next(context);
            }
            catch (Exception ex)
            {
                // If an exception is thrown, handle it here

                // Set the response status code to 500 Internal Server Error
                context.Response.StatusCode =
                StatusCodes.Status500InternalServerError;

                // Set the response content type to application/json
                context.Response.ContentType = "application/json";

                // Create a custom error response object
                var customErrorResponse = new
                {
                    Code = 500,
                    Message = "Internal Server Error Occurred",
                };
            }
        }
    }
}
```

```

        ExceptionDetails = ex.Message // Include exception message
for debugging purposes
    };

    // Log the Exception Details (logging not implemented here, but
this comment indicates where it should be done)

    // Serialize the custom error response object to JSON
    var responseJson = JsonSerializer.Serialize(customErrorResponse);

    // Write the JSON response to the HTTP response body
    await context.Response.WriteAsync(responseJson);
}
}
}
}
}

```

Register the Middleware Component into the Request Processing Pipeline:

After creating the middleware, you need to register it in the Program class. The Error Handling middleware should be one of the first components added to the pipeline:

```
app.UseMiddleware<ErrorHandlingMiddleware>();
```

Modifying the Employee Controller:

Next, modify the Employee Controller as follows. Here, you can see that we are not handling the exception that causes unhandled exceptions during program execution, which will be handled by the Exception Middleware component.

```

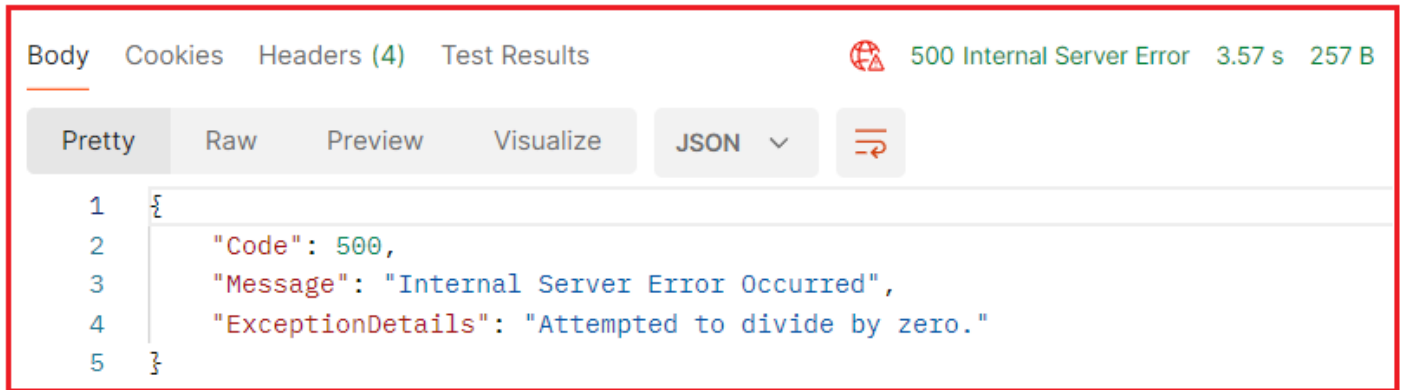
using Microsoft.AspNetCore.Mvc;

namespace ReturnTypesAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        public IActionResult GetEmployee()
        {
            // Your logic here
            int x = 10, y = 0;
            int z = x / y; //This statement will throw exception
            return Ok();
        }
    }
}

```

```
}  
}  
}
```

Now, save the changes, run the application, access the above endpoint, and you will get the following 500 responses:



```
{  
  "Code": 500,  
  "Message": "Internal Server Error Occurred",  
  "ExceptionDetails": "Attempted to divide by zero."  
}
```

How to Troubleshoot 500 Internal Server Error in ASP.NET Core Web API?

To Troubleshoot 500 Internal Server Error in ASP.NET Core Web API, we need to check the following:

- **Check Application Logs:** The first step in troubleshooting a 500 error is to look at the logs. ASP.NET Core uses a powerful logging API that can be configured to log detailed error information to various outputs such as console, file, or external monitoring services.
- **Debug Locally:** Run the application in a development environment with detailed error messages enabled. Ensure that the environment is set to Development and use the `UseDeveloperExceptionPage()` middleware, which will show detailed exceptions in the browser when an error occurs.
- **Use Exception Handling Middleware:** Implement global exception handling in your ASP.NET Core application using middleware such as `UseExceptionHandler()` in the request pipeline. This allows you to catch unhandled exceptions and log them or return a custom error response.
- **Review Dependency Configuration:** Ensure all services and dependencies are properly configured in the dependency injection container in the Program class. Misconfigured services can lead to runtime errors that might not be apparent at the start of the application.
- **Review Code for Exceptions:** Check your controllers, services, and any other components for places where exceptions might be thrown but not handled. Ensure that you have appropriate try-catch blocks where exceptions might occur.
- **Check External Dependencies:** Verify that any third-party services or APIs the application relies on are available and functioning correctly. Handle potential timeouts or errors from these services.

In the next article, I will discuss [how to Return the 501 Not Implemented Error HTTP Status Code in ASP.NET Core Web API](#) with Examples. In this article, I try to explain **how to Return 500 Internal**

Server Error HTTP Status Codes in ASP.NET Core Web API with Examples. I hope you enjoy this article, “How to Return 500 Internal Server Error HTTP Status Code in ASP.NET Core Web API.”



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

1 thought on “500 HTTP Status Code in ASP.NET Core Web API”



SAMEERA

FEBRUARY 21, 2025 AT 2:37 PM

thank you sir.

[Reply](#)

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information